

架构辩论赛记录

团队：暴风星云裂 | 项目：CampusHub | 日期：2026年4月29日

说明：团队实际为 3 名成员。为满足 P2 要求的 4 种 Prompt 策略，本次实验按 4 个项目角色记录，其中王胜晟兼任需求负责人和测试负责人。

1. 实验目标

通过不同 Prompt 策略向 AI 咨询 CampusHub 架构建议，比较 AI 输出质量差异，并由团队结合 P1 需求边界做最终架构决策。

核心判断标准：

- 是否符合学生项目 10 周内交付的约束。
- 是否遵守既定技术栈：Vue3、Java、Spring Boot、MySQL。
- 是否避免不必要中间件和商业化复杂架构。
- 是否覆盖订单状态一致性、权限安全、可维护性等质量属性。

2. Prompt 与 AI 回答记录

2.1 成员 A：直接询问推荐架构

成员/角色：王胜晟（需求负责人）

Prompt:

我们要做一个校园互助服务平台，推荐使用什么软件架构？

AI 回答摘要：

AI 推荐可以采用前后端分离架构，也提到如果后续规模扩大可以考虑微服务。前端可使用 Vue 或 React，后端可使用 Spring Boot，数据库可使用 MySQL。AI 还建议可以使用 Redis 做缓存、消息队列处理通知、对象存储保存图片。

输出质量评价：2/5

人工评价：

回答覆盖面较广，但缺少项目约束，回答比较笼统，没有特别贴合项目需求。

2.2 成员 B：提供详细项目约束后询问

成员/角色：王自宽（架构负责人）

Prompt:

我们是 3 人学生团队，10 周内完成 CampusHub 校园互助服务平台 MVP。功能包括学校邮箱注册登录、需求发布、任务大厅、接单与订单状态流转、订单内实时文字和图片聊天、评价信用、举报审核和基础后台。技术栈已确定为前端 Vue3、后端 Java Spring Boot、数据库 MySQL。不做在线支付、推荐算

法、语音/视频通话、群聊和位置共享，不希望引入 Redis、消息队列、网关等中间件。请推荐合适架构。

AI 回答摘要：

AI 推荐前后端分离的分层单体架构：Vue3 负责页面与状态管理，Spring Boot 提供 RESTful API 和 WebSocket 连接，后端按 Controller、Service、Repository 分层，并按用户、需求、订单、消息、评价、举报、后台等模块组织代码。MySQL 负责持久化，订单内聊天消息落库后通过 WebSocket 推送，系统通知可使用通知表和轮询，过期需求使用 Spring 定时任务。AI 明确指出微服务对 3 人团队和 10 周 MVP 过重。

输出质量评价： 5/5

人工评价：

该回答与项目约束高度匹配，能够主动压缩技术复杂度，保留了订单一致性、权限校验和可维护性等重点。

2.3 成员 C：要求比较多种方案优劣

成员/角色： 蔡利扬（开发负责人）

Prompt：

请从 MVC/分层单体、微服务、事件驱动三种架构中，为 CampusHub 进行对比。项目是学生 MVP，前端 Vue3，后端 Spring Boot，数据库 MySQL，不需要复杂中间件。请从开发复杂度、维护性、扩展性、性能、学习成本、10 周可行性对比，并给出最终推荐。

AI 回答摘要：

AI 列表比较了三种方案。分层单体开发复杂度最低、交付风险最低；微服务扩展性较好但需要服务治理、分布式事务和多服务部署；事件驱动适合通知和审计等异步场景，但需要消息队列和幂等机制。AI 最终推荐分层单体，并建议在代码层面保持模块边界，为未来拆分服务留余地。

输出质量评价： 4/5

人工评价：

对比结构清晰，适合直接转化为架构候选方案表。缺点是对“模块边界如何落到接口”说明较少，需要人工补充模块依赖图和 API 示例。

2.4 成员 D：让 AI 扮演架构师角色

成员/角色： 王胜晟（测试负责人）

Prompt：

请你扮演一名学生项目架构师，为 CampusHub 做架构评审。请重点检查：（1）订单状态一致性；（2）用户权限和隐私；（3）无需中间件时通知和搜索如何实现；（4）后续如果扩展怎么演进。请不要推荐超出学生团队能力的复杂方案。

AI 回答摘要：

AI 建议采用模块化单体，并把订单作为核心领域模块处理。订单状态变更必须通过统一的 OrderService 完成，使用数据库事务和状态枚举避免非法流转。权限方面使用 Spring Security + JWT，敏感信息前端脱敏。订单内

聊天使用 Spring WebSocket 负责实时推送，文字和图片消息都写入 order_message 表；系统通知使用 notification 表和前端轮询实现，搜索先用 MySQL 索引和 LIKE/全文索引。后续可从文件、通知、搜索等非核心模块逐步演进。

输出质量评价： 5/5

人工评价：

该回答质量最高，原因是 Prompt 明确要求检查质量属性和项目边界。它补足了前几个回答中对一致性、权限和演进路径的不足。

3. 团队辩论记录

讨论点	主要观点	结论
是否需要微服务	微服务利于扩展，但当前只有 3 人、10 周、200 并发目标，服务拆分会消耗大量开发和部署时间。	不采用微服务。
是否需要消息队列	接单、评价、举报等流程需要可追踪的一致性。通知量小，可用数据库通知表和轮询；订单内实时聊天只需要当前单体应用中的 WebSocket 推送。	不引入消息队列。
是否需要 Redis 缓存	当前性能目标可通过分页、索引、合理 SQL 达到，缓存会带来一致性和运维问题。	不引入 Redis。
后端模块如何划分	不能只按表写 CRUD，需要围绕用户、需求、订单、消息、评价、举报、后台等业务能力划分。	采用模块化单体，模块内分层。
订单一致性如何保证	订单状态是核心风险点，应集中在 OrderService 中处理，并使用事务和状态机式校验。	ADR 中单独记录。
前端是否需要后台单独工程	后台页面功能较轻，可以和前台共用 Vue3 工程，通过路由和权限控制区分。	首版采用单个前端工程。

4. 投票结果

成员/角色	投票方案	理由
王胜晟（需求负责人）	前后端分离的分层单体	最符合 P1 需求范围，能快速完成核心流程。
王自宽（架构负责人）	前后端分离的分层单体	技术栈明确，架构复杂度可控，便于写 ADR 和落地开发。
蔡利扬（开发负责人）	前后端分离的分层单体	代码组织清晰，开发调试成本低，适合 10 周实现。
王胜晟（测试负责人）	前后端分离的分层单体	单体架构更容易构造端到端测试和状态流转测试。

最终结论： 全票选择 前后端分离的分层单体架构。

5. 不选择其他方案的原因

5.1 不选择微服务

- 当前项目没有多团队并行维护需求。
- 多服务部署、鉴权传递、接口联调、日志追踪和故障定位会显著增加成本。
- 用户、需求、订单、评价、举报之间存在较强业务关联，拆分后会引入分布式一致性问题。
- 课程项目验收关注完整业务闭环，不关注服务治理能力。

5.2 不选择事件驱动作为主架构

- 事件驱动通常需要消息队列等中间件，与“中间件不需要”的项目约束冲突。
- 订单状态流转必须即时反馈给用户，不适合完全异步化。
- 事件驱动可作为后续局部优化，例如异步通知、操作审计，但首版不采用。

5.3 不选择纯后端 MVC 页面渲染

- P1 已确定前后端分离，前端 Vue3 更适合实现任务大厅、发布表单、后台管理等交互页面。
- RESTful API 有利于接口测试和前后端并行开发。

6. Prompt 策略反思

本次实验验证了 P1 反思日志中的改进计划：

1. **提供上下文锚点有效。** 成员 B 的 Prompt 附上团队人数、技术栈、功能范围和排除项，AI 输出明显更贴合项目。
2. **要求对比方案有效。** 成员 C 的 Prompt 使 AI 能形成候选方案对比表，便于团队讨论。
3. **要求审查质量属性有效。** 成员 D 的 Prompt 让 AI 关注订单一致性、权限和后续演进，输出比直接问更可用。
4. **反例约束必要。** 直接询问时 AI 容易推荐 Redis、消息队列、微服务等“看起来专业但不适合”的方案，需要人工明确排除。